Software Retirement Standard - (SRTS)

Software Retirement Standard - (SRTS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Retirement Governance

Standard ID: SLGA-SRTS-10

Origin ID: OOF-SLGA-SRTS

Version: 1.0

Status: Canonical Open

Effective Date: 15.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: UCL — Universal Canonical Language (English)

Canonical Definition

Software Retirement Standard (SRTS) is the Parent Standard within SLGA™ — Software Lifecycle Governance Architecture that governs how software products, versions, components, services, dependencies, and associated operational capabilities are identified, evaluated, prepared, transitioned, withdrawn, and formally retired from their applicable lifecycle and operational environments.

SRTS establishes software retirement as a governed transition from an

identifiable supported, maintained, deployed, or otherwise active software condition toward a defined retirement state in which the software's operational role, support obligations, dependencies, data relationships, authority conditions, and residual responsibilities have been appropriately resolved or transferred.

The standard distinguishes the decision to retire software from the actual

withdrawal of its capabilities, the termination of support from the termination of operation, the removal of a software artifact from the elimination of its dependencies, and administrative retirement from verified operational retirement.

It requires retirement decisions and activities to preserve the identity of

the retiring software, the reasons for retirement, affected users and systems, operational dependencies, migration requirements, data continuity, support boundaries, transition conditions, withdrawal evidence, and remaining obligations.

SRTS governs planned retirement, end-of-support transitions, replacement

and migration, obsolete software withdrawal, dependency discontinuation, unsupported component removal, and retirement resulting from security, operational, economic, architectural, or external requirements.

It recognizes that software may cease to receive support while remaining operational, that retired software may retain active dependencies or historical obligations, and that retirement cannot be considered complete merely because a product is no longer distributed or maintained.

Its principal governance output is the Software Retirement Governance Profile.

Operational Role

SRTS establishes the governance framework for the final transition of

software from active lifecycle participation toward an explicitly defined retirement condition.

It governs:

• Software retirement identification

• Retirement purpose and justification

• Retirement object identity

• Retirement scope and boundaries

• Retirement readiness

• Retirement planning

• Retirement decision authority

• End-of-support conditions

• End-of-maintenance conditions

• End-of-operation conditions

• Software replacement

• Migration requirements

• Dependency identification

• Residual dependency control

• Data continuity

• Interface continuity

• User and external stakeholder effects

• Operational transition

• Security implications

• Retirement execution

• Software withdrawal

• Retirement verification

• Residual obligations

• Retirement exceptions

• Retirement reassessment

• Retirement closure

• Historical and evidentiary continuity

SRTS does not require all software to follow an identical retirement pathway.

Its governance requirements adapt to the software's actual operational

role, dependencies, criticality, and retirement consequences.

Core Question

How can software be legitimately, safely, and verifiably retired when its operational use, support obligations, dependencies, data, users, authority relationships, and residual responsibilities may continue beyond the decision to discontinue it?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SRTS is the tenth Parent Standard within SLGA™.

It governs the retirement stage of the software lifecycle while recognizing

that retirement may trigger renewed activity within earlier lifecycle mechanisms.

A retirement plan may require new software requirements, migration software creation, dependency changes, configuration modifications, security verification, release preparation, deployment transitions, or change management.

Software retirement is therefore not necessarily a single terminal action.

It may be a sequence of governed transitions involving multiple software objects, environments, actors, and residual obligations.

• Governed Space

Software Retirement Governance

This governed space concerns the controlled discontinuation, withdrawal,

replacement, migration, or retirement of identifiable software objects and their associated operational roles.

It begins when a software object becomes subject to a retirement proposal, requirement, or decision.

It includes the evaluation of retirement feasibility, consequences,

dependencies, migration pathways, operational continuity, support obligations, and retirement conditions.

It continues through transition, withdrawal, verification, residual dependency management, and formal closure.

The governed space remains applicable where a software object has been declared retired but unresolved operational dependencies, data relationships, or obligations continue to exist.

Governed Object

The primary Governed Object is:

A Software Retirement Case representing the evidence-supported

relationship between an identifiable software object, its existing operational and support conditions, retirement purpose, affected dependencies, required transition activities, intended retirement state, actual withdrawal results, residual obligations, and final retirement disposition.

The governed object must preserve, where applicable:

Retirement Case ID

Software Object ID

Software Product or Component Reference

Current Lifecycle State

Current Operational State

Support Status

Retirement Purpose

Retirement Scope

Affected Dependencies

Affected Users and Systems

Migration Requirements

Data Continuity Requirements

Withdrawal Conditions

Applicable Authority

Retirement Evidence

Residual Obligations

Retirement Status

Retirement Version

Scope

SRTS governs:

Software Product Retirement

Software Version Retirement

Software Component Retirement

Software Service Retirement

Dependency Retirement

Software Interface Retirement

Firmware Retirement

Software Capability Withdrawal

End-of-Support Management

End-of-Maintenance Management

Retirement Planning

Migration Governance

Replacement Transition

Residual Dependency Governance

Retirement Verification

Retirement Closure

The standard applies to software operating in development, testing,

production, embedded, industrial, autonomous, and distributed environments.

It does not govern the physical disposal of hardware except where such disposal materially affects software retirement conditions or requires interaction with another applicable governance mechanism.

Foundational Principle

Software retirement is complete only when the software's intended operational withdrawal has been established and its remaining dependencies, data relationships, support conditions, and obligations have been appropriately resolved, transferred, or explicitly retained under governance.

A retirement announcement does not establish operational retirement.

End of support does not establish end of use.

Artifact deletion does not establish dependency elimination.

Migration completion does not automatically establish retirement closure.

Software Discontinued ≠ Software Retired.

End of Support ≠ End of Operation.

Artifact Removed ≠ Operational Dependency Eliminated.

Software Retirement Identity

Every governed retirement case must have a stable identity.

Relevant elements include:

Retirement Case ID

Software Object ID

Product or Component Name

Version Reference

Deployment Reference

Retirement Origin

Responsible Governance Function

Creation Time

Retirement Status

Evidence Reference

The retirement identity must remain stable throughout planning, migration, withdrawal, verification, and closure.

Retirement Object Identification

SRTS must identify the software object subject to retirement.

Potential objects include:

Application

Software Product

Software Service

Software Version

Source Component

Library

Dependency

Interface

Runtime Component

Firmware

Automated Workflow

AI Agent Software Component

Embedded Software Capability

The identity must be sufficiently precise to distinguish the retiring object from related software that remains active.

Retirement Origin

Retirement may originate from:

Technology Obsolescence

End of Vendor Support

Security Exposure

Maintenance Infeasibility

Replacement Availability

Operational Redundancy

Architectural Transition

Infrastructure Change

External Requirement

Economic Sustainability

Software Consolidation

System Decommissioning

Safety or Reliability Concerns

Dependency Discontinuation

The retirement origin must remain distinguishable from the retirement decision.

Retirement Purpose

Every retirement case must establish its purpose.

Potential purposes include:

Eliminate Unsupported Software

Reduce Security Exposure

Replace Obsolete Functionality

Remove Redundant Components

Consolidate Software Services

Improve Operational Sustainability

Complete Technology Migration

Terminate an Unnecessary Capability

Remove an Unacceptable Operational Risk

Fulfill an External Obligation

The purpose must identify the intended retirement outcome rather than merely the planned technical action.

Retirement Justification

Retirement justification must establish why the software should cease its existing lifecycle or operational role.

Relevant elements include:

Current Software Condition

Support Availability

Operational Necessity

Security Exposure

Maintenance Requirements

Replacement Feasibility

Dependency Consequences

Economic Considerations

External Obligations

Risk of Continued Operation

The justification must be supported by applicable evidence and remain open to reassessment when material conditions change.

Current Software State

SRTS must establish the relevant current state of the software.

Relevant elements include:

Software Version

Artifact Identity

Configuration

Deployment Environments

Operational Function

Support Status

Maintenance Status

Known Dependencies

Data Relationships

Active Users

Security Conditions

Known Restrictions

Retirement planning must not assume that a software object is inactive merely because its maintenance or distribution has stopped.

Intended Retirement State

Every retirement case must define the intended resulting condition.

Potential states include:

No Longer Distributed

No Longer Supported

No Longer Maintained

No Longer Deployed

No Longer Executable in the Governed Environment

Operationally Replaced

Functionality Migrated

Dependencies Removed

Historical Artifact Retained Without Operational Use

Retired With Explicit Residual Obligations

The intended retirement state must be sufficiently defined to support verification.

Retirement State Distinction

SRTS must distinguish among:

Retirement Proposed

Retirement Approved

End of Support

End of Maintenance

Operational Withdrawal Planned

Operational Withdrawal Active

Operational Withdrawal Verified

Residual Obligations Active

Retirement Closed

These states must not be collapsed into a single retirement flag.

Retirement Scope

Retirement scope must identify what is being withdrawn or discontinued.

Relevant dimensions include:

Software Products

Software Versions

Components

Services

Interfaces

Dependencies

Operational Environments

Users

Data Relationships

Support Obligations

Maintenance Obligations

External Integrations

The scope must distinguish complete retirement from partial or environment-specific retirement.

Retirement Boundary

The retirement boundary identifies what the retirement case includes and

excludes.

Relevant boundaries include:

Software Object Boundary

Version Boundary

Environment Boundary

Operational Boundary

Dependency Boundary

Data Boundary

Interface Boundary

Authority Boundary

Support Boundary

Historical Evidence Boundary

An exclusion must not conceal a dependency or obligation materially affected by retirement.

Partial Software Retirement

A software object may be retired from one environment while remaining active elsewhere.

Examples include:

Production Retirement With Historical Archive Retention

Retirement of One Software Version

Withdrawal From One Geographic Region

Removal From One Customer Environment

Discontinuation of One Interface

Retirement of One Firmware Variant

Partial retirement must preserve the identity and continuing governance status of software that remains active.

Retirement Significance

SRTS must evaluate the governance significance of retirement.

Relevant factors include:

Operational Criticality

Affected Users

Dependency Scope

Data Continuity

Security Exposure

Migration Complexity

Replacement Availability

Recovery Requirements

External Obligations

Physical-System Consequences

Retirement significance must not be determined solely by the age or monetary value of the software.

Retirement Feasibility

Retirement feasibility concerns whether the intended withdrawal can be achieved under applicable conditions.

Relevant elements include:

Replacement Availability

Migration Capability

Dependency Removal Feasibility

Data Accessibility

Operational Continuity

Required Expertise

Required Resources

External System Compatibility

Applicable Authority

Verification Capability

A justified retirement objective does not establish that immediate withdrawal is feasible.

Retirement Alternatives

Where material consequences justify comparison, SRTS must consider applicable alternatives.

Potential alternatives include:

Immediate Retirement

Phased Retirement

Temporary Continued Operation

Restricted Operation

Software Replacement

Component Isolation

Dependency Substitution

Migration

Extended Support

Controlled Decommissioning

Alternatives must be assessed against the retirement purpose and applicable operational conditions.

Retirement Decision

A retirement decision must preserve:

Decision Reference

Retiring Software Object

Retirement Purpose

Approved Scope

Intended Retirement State

Applicable Conditions

Required Transition Activities

Residual Obligation Requirements

Competent Authority

Decision Time

A retirement decision does not itself establish that withdrawal has occurred.

Retirement Authority

SRTS must identify the competent authority for material retirement decisions.

Relevant elements include:

Software Ownership

Operational Responsibility

Support Responsibility

Retirement Decision Authority

Migration Authority

Deployment or Withdrawal Authority

Data Governance Authority

External Approval Requirements

Delegation Conditions

The authority to stop supporting software does not automatically establish authority to terminate every system that depends on it.

Retirement Planning

Retirement planning must establish the pathway from the current software state to the intended retirement state.

Relevant elements include:

Retirement Scope

Affected Software Objects

Dependency Assessment

Support Conditions

Migration Requirements

Transition Sequence

Required Resources

Operational Restrictions

Verification Requirements

Recovery or Contingency Conditions

Residual Obligations

The retirement plan must remain traceable to the applicable decision and evidence.

Retirement Readiness

Retirement readiness concerns whether the conditions necessary to begin or

complete retirement have been established.

Relevant elements include:

Retirement Decision Applicable

Dependencies Identified

Affected Users Identified

Migration Pathway Established

Data Continuity Addressed

Required Resources Available

Operational Restrictions Established

Verification Plan Available

Residual Obligations Identified

Readiness must remain distinct from actual retirement completion.

Retirement Sequence

Software retirement may require a defined sequence.

A typical pathway may include:

Retirement Decision → Dependency Assessment → Migration Preparation →

Replacement Verification → Operational Transition → Software Withdrawal → Residual Dependency Assessment → Retirement Closure

The sequence must adapt to the actual software environment.

Not every retirement requires replacement or migration.

Mandatory dependencies and verification conditions must nevertheless be preserved.

End-of-Support Governance

SRTS governs the transition of software into an end-of-support condition.

Relevant elements include:

Support Provider

Supported Software Version

Support Scope

Support End Condition

Security Update Availability

Technical Assistance Availability

Continuing Operational Use

Known Restrictions

Affected Stakeholders

Residual Responsibilities

End of support must not be treated as proof that the software is no longer operational.

End-of-Maintenance Governance

End of maintenance concerns the cessation of ordinary corrective or

improvement activities.

Relevant elements include:

Maintenance Scope

Maintenance End Condition

Remaining Security Obligations

Critical Correction Exceptions

Continued Operational Use

Maintenance Dependencies

Replacement or Retirement Plan

Applicable Restrictions

End of maintenance and end of support may occur at different times.

Unsupported Software Operation

Software may remain operational after support or maintenance ends.

SRTS must identify:

Unsupported Software Identity

Operational Environment

Continuing Business or Technical Need

Known Vulnerabilities

Dependency Exposure

Operational Restrictions

Responsible Function

Replacement or Retirement Conditions

Review Requirements

Unsupported operation must remain an explicit governance condition rather than an unrecognized lifecycle state.

Support Transition Communication

Material support changes may require communication to affected parties.

Relevant elements include:

Affected Users

Operational Owners

Software Maintainers

External Customers

Integration Partners

Support End Conditions

Required User Actions

Migration Availability

Operational Restrictions

Communication requirements must reflect applicable responsibilities and external obligations.

Software Replacement Governance

Retirement may involve replacing an existing software object.

Relevant elements include:

Retiring Software

Replacement Software

Replacement Purpose

Functional Equivalence

Compatibility Requirements

Security Conditions

Data Continuity

Operational Readiness

Verification Evidence

Replacement availability does not establish replacement suitability.

Replacement Equivalence

Where replacement software is intended to preserve existing capabilities, SRTS must identify the required equivalence conditions.

Potential dimensions include:

Functional Equivalence

Interface Compatibility

Data Compatibility

Security Requirements

Performance Requirements

Operational Reliability

Recovery Capability

External System Compatibility

User Requirements

Equivalence must be evaluated against the actual obligations of the retiring software.

Software Migration Governance

Migration may be required to transfer functions, users, data, or operational dependencies.

Relevant elements include:

Source Software

Destination Software

Migration Scope

Migration Method

Affected Users

Data Transfer Requirements

Compatibility Conditions

Transition Sequence

Verification Requirements

Fallback Conditions

Migration completion must remain distinct from retirement closure.

Migration Identity and Traceability

Every material migration must preserve:

Migration Reference

Source Software Identity

Destination Software Identity

Affected Data or Functions

Migration Actor

Migration Time

Execution Evidence

Verification Evidence

Known Limitations

Residual Obligations

Traceability must support reconstruction of what was transferred and what remained behind.

Migration Readiness

Migration readiness may depend on:

Destination Availability

Functional Compatibility

Data Compatibility

Required Interfaces

Security Conditions

Operational Resources

Verification Capability

Recovery Feasibility

Affected Stakeholder Preparation

Applicable Authority

Migration must not proceed solely because the retiring software has reached an announced support deadline.

Migration Verification

SRTS must establish whether the required migration outcome has been achieved.

Relevant elements include:

Transferred Functions

Transferred Data

Affected Users

Destination Behavior

Compatibility Results

Data Integrity

Operational Continuity

Untransferred Objects

Migration Errors

Verification Evidence

A migration may be technically complete while material operational obligations remain unresolved.

Data Continuity

Retirement must preserve applicable data continuity requirements.

Relevant elements include:

Data Ownership

Data Location

Data Accessibility

Data Integrity

Data Migration

Data Retention

Data Deletion Requirements

Historical Records

Backup Dependencies

Recovery Requirements

Software retirement must not silently eliminate data that remains subject to legitimate retention or access obligations.

Data Retention and Disposal Boundary

SRTS must distinguish software retirement from data retention and disposal.

Potential conditions include:

Software Retired — Data Retained

Software Retired — Data Migrated

Software Retired — Data Archived

Software Retired — Data Disposal Pending

Software Retired — Data Disposal Verified

Data retention and deletion must follow applicable specialized governance and legal requirements.

SRTS preserves their retirement-related dependencies and completion

evidence.

Interface Continuity

Retiring software may provide interfaces used by other systems.

Relevant elements include:

Interface Identity

Dependent Consumers

Communication Protocols

Data Exchange Requirements

Replacement Interfaces

Compatibility Conditions

Transition Timing

Withdrawal Evidence

An interface must not be withdrawn without appropriate treatment of material dependent systems.

Dependency Identification

SRTS must identify dependencies associated with the retiring software.

Relevant categories include:

Direct Software Dependencies

Transitive Dependencies

Runtime Dependencies

Infrastructure Dependencies

External Service Dependencies

Data Dependencies

Interface Dependencies

Operational Process Dependencies

Physical-System Dependencies

Dependency assessment must include both what the retiring software depends on and what depends on the retiring software.

Upstream Dependency Governance

Upstream dependencies are resources or systems required by the retiring software.

Relevant elements include:

Required Libraries

External Services

Runtime Platforms

Databases

Infrastructure Components

Authentication Services

Configuration Services

Operational Support Systems

Retirement must identify whether upstream resources can also be withdrawn or remain necessary for other software.

Downstream Dependency Governance

Downstream dependencies concern systems, users, or processes relying on the

retiring software.

Relevant elements include:

Dependent Applications

External Integrations

Automated Workflows

Business Processes

Data Consumers

Operational Services

Robotic Systems

Industrial Equipment

External Customers

Downstream dependencies must not be assumed eliminated merely because the retiring software is no longer actively maintained.

Residual Dependency

A residual dependency is a continuing relationship or obligation associated with software that has entered or completed part of its retirement process.

Potential residual dependencies include:

Legacy Interface Consumer

Historical Data Access

Unmigrated Function

Archived Software Execution Requirement

Recovery Dependency

External Service Contract

Unsupported Operational Component

Physical Equipment Dependency

Residual dependencies must remain identifiable and governed.

Residual Dependency Classification

SRTS may classify residual dependencies as:

Resolved

Transferred

Replaced

Isolated

Restricted

Accepted Under Applicable Governance

Pending Migration

Pending Removal

Unknown

Conflicted

Classification must preserve the supporting evidence and applicable responsibility.

Residual Dependency Responsibility

Every material residual dependency must have an identifiable responsibility or control reference.

Relevant elements include:

Dependency ID

Retiring Software Reference

Dependent Object

Responsible Function

Required Action

Applicable Authority

Operational Restriction

Resolution Condition

Evidence

A retirement case must not be closed as fully resolved while material residual dependencies remain unaccounted for.

Security Implications of Retirement

Retirement may alter security conditions.

Relevant elements include:

Unsupported Vulnerabilities

Residual Credentials

Service Accounts

Access Permissions

Exposed Interfaces

Unmaintained Components

Archived Software

Data Exposure

Retained Secrets

Monitoring Requirements

Retirement must not leave unnecessary active security capabilities without an appropriate governance basis.

Access and Credential Withdrawal

Where applicable, retirement must address:

Software Accounts

Service Credentials

API Keys

Certificates

Agent Permissions

Automated Execution Rights

Administrative Access

External Integration Credentials

Machine Identities

Withdrawal must account for shared credentials and dependencies so that unrelated systems are not unintentionally disrupted.

Operational Continuity

SRTS must assess whether retirement affects essential operational capabilities.

Relevant elements include:

Critical Functions

Service Availability

Replacement Readiness

Fallback Capability

Data Accessibility

External Integrations

Operational Dependencies

Recovery Conditions

Affected Users

Retirement must not create unacceptable operational interruption merely because the software is obsolete or unsupported.

Retirement of Autonomous Software

Retirement of autonomous software may require additional governance.

Relevant elements include:

Agent Identity

Assigned Objectives

Active Tasks

Delegated Authority

Tool Permissions

Memory and Data References

External Actions

Pending Commitments

Human Override

Successor System

An autonomous agent must not be considered retired solely because its primary execution process has stopped.

Active permissions, delegated actions, and unresolved obligations may remain.

Autonomous Authority Termination

Where autonomous software is retired, SRTS must identify whether applicable execution authority has been terminated or transferred.

Relevant elements include:

Agent Identity

Authority Reference

Delegated Permissions

Tool Access

External System Access

Scheduled Actions

Persistent Credentials

Successor Authority

Termination Evidence

Residual Obligations

Agent Execution Stopped ≠ Agent Authority Terminated.

AI Model and Service Retirement

Where an AI model or service is retired, SRTS must identify relevant continuing relationships.

Potential elements include:

Model or Service Identity

Deployed Versions

Dependent Applications

API Consumers

Stored Outputs

Operational Workflows

Model-Specific Configuration

External Service Contracts

Replacement Compatibility

Residual Monitoring Obligations

The retirement of a model endpoint does not automatically retire systems or decisions dependent on its outputs.

Embedded and Industrial Software Retirement

Embedded software may remain operational within physical equipment long after active maintenance ends.

Relevant elements include:

Firmware Identity

Device Identity

Installed Version

Operational Function

Physical Safety Conditions

Replacement Feasibility

Maintenance Availability

Recovery Requirements

Equipment Dependencies

Retirement Restrictions

Software retirement may require coordination with physical asset governance.

Retirement Execution

Retirement execution concerns the activities performed to establish the

intended retirement state.

Potential activities include:

Service Withdrawal

Software Deactivation

Artifact Removal

Interface Shutdown

Dependency Removal

Configuration Removal

Credential Revocation

Data Migration

Operational Transfer

Monitoring Termination or Transfer

Execution must remain traceable to the approved retirement scope and applicable authority.

Withdrawal Control

Withdrawal must be governed according to the affected software and operational conditions.

Relevant elements include:

Withdrawal Object

Withdrawal Environment

Required Preconditions

Responsible Actor

Execution Authority

Withdrawal Method

Operational Restrictions

Verification Requirements

Failure Response

Execution Evidence

A withdrawal action must not exceed the applicable retirement authority.

Withdrawal Verification

SRTS must verify whether the intended withdrawal actually occurred.

Relevant elements include:

Software No Longer Executing

Service No Longer Available Where Intended

Artifact Removed or Disabled

Interface Withdrawn

Credentials Revoked

Dependent Systems Transitioned

Operational Monitoring Updated

Residual Execution Paths Identified

Verification must correspond to the defined retirement scope.

Retirement Evidence

SRTS must preserve evidence supporting material retirement conclusions.

Potential evidence includes:

Retirement Decision

Software Inventory

Dependency Assessment

Support Records

Migration Plan

Migration Results

Withdrawal Records

Operational Monitoring

Security Verification

Data Continuity Evidence

Residual Dependency Register

Closure Decision

Evidence must remain attributable to the software object, retirement case, and relevant operational conditions.

Retirement Evidence Applicability

Retirement evidence must be evaluated against:

Software Identity

Software Version

Environment

Operational State

Dependency State

Migration Scope

Withdrawal Scope

Evidence Time

Known Limitations

Evidence that one software instance was withdrawn does not establish that every instance or dependent service has been retired.

Retirement Monitoring

Some retirement activities require continued monitoring.

Potential monitoring concerns include:

Residual Software Execution

Legacy Interface Calls

Unexpected Dependency Use

Migration Failures

Security Exposure

Operational Disruption

Data Access Requirements

Replacement Performance

Unresolved Retirement Obligations

Monitoring duration and conditions must reflect the actual retirement risks and dependencies.

Retirement Failure

Retirement failure may occur when:

Required Migration Fails

Critical Dependencies Remain

Software Cannot Be Withdrawn

Operational Continuity Is Compromised

Data Becomes Inaccessible

Replacement Software Is Incompatible

Security Conditions Become Unacceptable

Required Verification Cannot Be Completed

Retirement Authority Is Insufficient

Failure must trigger appropriate reassessment, restriction, recovery, or alternative retirement processing.

Retirement Deviation

A retirement deviation occurs when actual retirement activities differ materially from the established plan or authorized scope.

Examples include:

Unplanned Software Withdrawal

Incomplete Migration

Unexpected Service Dependency

Unapproved Data Deletion

Unauthorized Credential Revocation

Changed Replacement Software

Unverified Interface Shutdown

Residual Production Execution

Material deviations must remain identifiable and subject to reassessment.

Retirement Reassessment

Retirement reassessment may be required when:

Dependency Relationships Change

Migration Conditions Change

Replacement Software Fails

New Operational Requirements Emerge

Security Conditions Change

Additional Software Instances Are Discovered

Data Continuity Requirements Change

Withdrawal Verification Fails

Residual Obligations Increase

Reassessment must preserve the earlier retirement plan and identify which conclusions remain applicable.

Retirement Exception Governance

An exception may be required where ordinary retirement requirements cannot be satisfied under current conditions.

Relevant elements include:

Exception ID

Affected Requirement

Reason

Applicable Authority

Alternative Control

Operational Restriction

Residual Risk

Validity Condition

Required Follow-up

Evidence

An exception must not be inferred from incomplete records or ignored dependencies.

Retirement Continuity

SRTS must preserve continuity across:

Retirement Planning

Support Termination

Migration

Replacement

Withdrawal

Operational Transition

Responsibility Transfer

Residual Dependency Management

Retirement Closure

Post-Retirement Discovery

Historical decisions, evidence, and unresolved obligations must remain reconstructable.

Retirement Closure

Retirement closure is the evidence-supported determination that the intended retirement disposition has been established and the applicable retirement obligations have been satisfied or legitimately retained under explicit governance.

Closure must preserve:

Retired Software Identity

Intended Retirement State

Actual Retirement State

Migration Outcome

Withdrawal Verification

Residual Dependencies

Data Continuity

Support Status

Operational Restrictions

Remaining Obligations

Closure Authority

Closure Evidence

Retirement closure must not be confused with deletion of a software record.

Retirement Closure Conditions

Successful retirement closure requires, as applicable:

Retirement Scope Established

Retirement Decision Applicable

Intended Withdrawal Verified

Required Migration Completed

Operational Continuity Addressed

Material Dependencies Resolved or Governed

Data Obligations Addressed

Access and Authority Conditions Addressed

Required Verification Completed

Residual Obligations Preserved

Competent Closure Decision Recorded

Where these conditions are not satisfied, the retirement case must retain an appropriate unresolved or qualified classification.

Retirement Finality

Retirement finality concerns whether the software has reached its intended

retirement condition under applicable governance requirements.

SRTS distinguishes:

Support Finality

Maintenance Finality

Distribution Finality

Operational Finality

Dependency Finality

Data Obligation Finality

Administrative Finality

These conditions may occur at different times.

A software product may be operationally retired while historical data and evidentiary obligations remain active.

Retirement Governance Classification

SRTS recognizes the following governance classifications:

Retirement Proposed

Retirement Under Assessment

Retirement Approved

Retirement Planning Active

Retirement Readiness Pending

Ready for Retirement

End of Support

End of Maintenance

Unsupported but Operational

Migration Active

Migration Verified

Withdrawal Active

Withdrawal Verified

Partially Retired

Retired With Residual Obligations

Retirement Completed

Retirement Closure Pending

Retirement Closed

Retirement Blocked

Retirement Failed

Reassessment Required

Exception Active

Unknown

Conflicted

Each classification must remain associated with its applicable software scope, evidence, and time.

Core Modules

SRTS is implemented through five Core Modules. 69.1 RPM — Retirement Planning Module

Governed Space: Software Retirement Planning Governance

Governs retirement initiation, purpose, feasibility, scope, affected

software objects, operational conditions, retirement readiness, required transition activities, sequencing, and retirement planning evidence.

Principal Output: Software Retirement Plan Profile

69.2 ESMM — End-of-Support Management Module

Governed Space: Software End-of-Support Governance

Governs support and maintenance termination conditions, unsupported

software operation, continuing obligations, affected stakeholders, operational restrictions, support transition evidence, and end-of-support continuity.

Principal Output: Software End-of-Support Profile

69.3 SMM — Software Migration Module

Governed Space: Software Migration Governance

Governs the transition of software functions, users, data relationships,

operational capabilities, and relevant dependencies from retiring software to replacement or successor environments.

Principal Output: Software Migration Governance Profile

69.4 RDMM — Residual Dependency Management Module

Governed Space: Software Retirement Residual Dependency Governance

Governs the identification, assessment, classification, ownership,

restriction, transfer, replacement, and resolution of dependencies that remain associated with software undergoing or having undergone retirement.

Principal Output: Software Residual Dependency Profile

69.5 RCCLM — Retirement Closure Control Module

Governed Space: Software Retirement Closure Governance

Governs retirement outcome verification, intended–actual retirement

alignment, withdrawal evidence, migration and dependency reconciliation, residual obligations, retirement finality, and legitimate closure disposition.

Principal Output: Software Retirement Closure Profile

The final canonical names, boundaries, and outputs of individual modules

are established through their respective full canonical publications.

Parent Standard Governance Flow

Retirement Origin → Software Identification → Retirement Purpose → Current

Software State → Retirement Scope → Retirement Impact → Retirement Feasibility → Retirement Decision → Retirement Planning → End-of-Support Conditions → Dependency Assessment → Migration Requirements → Transition Coordination → Retirement Readiness → Governed Withdrawal → Withdrawal Verification → Residual Dependency Assessment → Residual Obligation Reconciliation → Retirement Closure

Where replacement is required:

Retiring Software → Replacement Identification → Suitability Assessment →

Migration → Compatibility Verification → Operational Transition → Withdrawal

Where unsupported software remains operational:

Support Termination → Continued Operational Use → Risk and Dependency

Assessment → Operational Restrictions → Retirement or Replacement Pathway

Where retirement fails:

Retirement Activity → Failure or Deviation → Actual State Assessment →

Reassessment → Revised Retirement Pathway

Where residual dependencies remain:

Withdrawal → Residual Dependency Identification → Responsibility →

Resolution / Transfer / Restriction → Closure Assessment

The governance flow must remain adaptable to the actual retirement context.

Governance Requirements

A conforming SRTS implementation must satisfy the following requirements. Retirement Identity and Decision

• Establish a stable retirement case identity

• Identify the software object subject to retirement

• Preserve the current lifecycle and operational state

• Establish the retirement origin

• Define the retirement purpose

• Preserve retirement justification

• Define the intended retirement state

• Identify competent retirement authority

Scope and Planning

• Establish retirement scope and boundaries

• Identify affected software versions and environments

- Evaluate retirement significance
- Evaluate retirement feasibility
- Identify applicable alternatives
- Establish retirement planning requirements
- Identify retirement readiness conditions
- Preserve required transition sequencing

Support and Maintenance

- Identify current support conditions
- Establish end-of-support requirements
- Distinguish support termination from operational withdrawal
- Identify end-of-maintenance conditions
- Govern continued operation of unsupported software
- Identify affected stakeholders
- Preserve material support transition communication
- Establish applicable operational restrictions

Migration and Dependencies

- Identify required replacement or migration activities
- Establish migration scope and conditions
- Preserve migration identity and traceability
- Verify material migration outcomes
- Identify upstream dependencies
- Identify downstream dependencies
- Identify residual dependencies
- Establish responsibility for residual dependencies
- Preserve data continuity requirements
- Identify interface transition requirements

Withdrawal and Verification

- Establish authorized withdrawal activities
- Preserve execution authority boundaries
- Identify required withdrawal preconditions
- Preserve withdrawal evidence
- Verify the intended retirement state
- Identify residual software execution
- Preserve security and access withdrawal requirements

• Identify material retirement deviations

• Establish reassessment triggers

• Preserve applicable retirement exceptions

Closure and Continuity

• Reconcile migration obligations

• Reconcile residual dependency obligations

• Reconcile data and support obligations

• Identify unresolved retirement conditions

• Establish retirement closure eligibility

• Preserve competent closure authority

• Establish evidence-supported retirement classification

• Preserve historical retirement continuity

• Maintain responsibility for residual obligations

• Prevent administrative retirement from substituting for verified operational withdrawal.

Governance Outputs

The principal output is:

Software Retirement Governance Profile

Supporting outputs may include:

Software Retirement Case Identity

Retirement Purpose Record

Retirement Justification

Current Software State Profile

Intended Retirement State Profile

Retirement Scope Record

Retirement Impact Assessment

Retirement Feasibility Assessment

Retirement Decision Record

Software Retirement Plan Profile

Software End-of-Support Profile

Unsupported Software Operation Register

Software Migration Governance Profile

Replacement Compatibility Record

Data Continuity Record

Software Dependency Retirement Map

Software Residual Dependency Profile

Withdrawal Execution Record

Withdrawal Verification Record

Retirement Evidence Register

Retirement Exception Register

Residual Retirement Obligation Register

Software Retirement Closure Profile

Governance Evidence

SRTS requires material retirement conclusions to remain supported by

evidence appropriate to the software object and operational conditions.

Relevant evidence may include:

Software Inventory

Version and Artifact Records

Support Agreements

Maintenance Records

Security Assessments

Dependency Maps

Migration Plans

Migration Verification

Operational Monitoring

Withdrawal Logs

Access Revocation Records

Data Continuity Evidence

Replacement Verification

Retirement Decisions

Residual Dependency Records

Closure Records

Evidence must preserve:

Source

Time

Software Identity

Applicable Scope

Collection or Verification Method

Known Limitations

Associated Decision

Evidence concerning one software instance or environment must not automatically establish retirement of every related instance.

Governance Continuity

SRTS must preserve retirement governance continuity when:

Software Ownership Changes

Support Providers Change

Operational Responsibility Transfers

Migration Is Interrupted

Replacement Software Changes

Retirement Scope Expands

Dependencies Are Discovered

Unsupported Software Remains Active

Withdrawal Fails

Residual Obligations Are Transferred

Continuity requires the retirement case, software identity, decisions,

evidence, affected dependencies, and outstanding responsibilities to remain reconstructable.

Cross-Architecture and Lifecycle Boundary

SRTS governs retirement-specific lifecycle transitions.

It does not replace the governance of software requirements, creation, dependencies, configuration, quality, security, release, deployment, or change management. SRS — Software Requirements Standard

Governs requirements affecting retirement, replacement, and migration. SCS — Software Creation Standard

Governs new software artifacts created for migration, replacement, or

retirement support. SDS — Software Dependency Standard

Governs dependency identity, suitability, introduction, change, and

continuity. SCFS — Software Configuration Standard

Governs configuration and artifact changes associated with retirement. SQS — Software Quality Standard

Governs quality verification of replacement or modified software. SSS — Software Security Standard

Governs security requirements, exposure, testing, vulnerabilities, and

remediation. SRLS — Software Release Standard

Governs release-related activities required for replacement or migration

software. SDPS — Software Deployment Standard

Governs deployment and operational transition activities. SCMS — Software Change Management Standard

Governs identifiable software changes initiated or required by retirement. SLCS — Software Lifecycle Control Standard

Governs lifecycle state consistency, transition control, exceptions,

authority transfer, and continuity.

SRTS may also interact with applicable OOF® identity, authority, accountability, operational reality, autonomous systems, data, and liability governance references.

These references remain independently applicable within their respective governed spaces.

Practical Applications

Enterprise Software Portfolios

Governing retirement of obsolete applications, redundant systems,

unsupported versions, and legacy services.

Cloud and Distributed Infrastructure

Managing service withdrawal, interface discontinuation, replacement

deployment, and residual dependencies across interconnected systems.

Software Supply Chains

Controlling retirement of outdated libraries, unsupported components, and

dependencies that remain embedded in active products.

AI Systems and Autonomous Agents

Governing model and agent retirement, termination of delegated permissions,

transition of active tasks, and continuing operational obligations.

Industrial and Embedded Software

Managing firmware retirement, equipment compatibility, operational

restrictions, and continued use of unsupported software within physical systems.

Data-Intensive Systems

Preserving data accessibility, retention, migration, and evidentiary

continuity after software withdrawal.

Critical Infrastructure

Establishing controlled retirement pathways where operational continuity, replacement feasibility, and residual dependencies materially affect safety or essential services.

Critical Distinctions

Software Discontinued ≠ Software Retired.

Retirement Announced ≠ Retirement Executed.

Retirement Approved ≠ Retirement Verified.

End of Support ≠ End of Operation.

End of Maintenance ≠ End of Support.

Unsupported Software ≠ Inactive Software.

Artifact Removed ≠ Operational Dependency Eliminated.

Software Replaced ≠ Replacement Verified.

Migration Executed ≠ Migration Outcome Verified.

Migration Completed ≠ Retirement Closed.

Software Deactivated ≠ All Execution Paths Eliminated.

Service Withdrawn ≠ Data Obligations Eliminated.

Upstream Dependency Removed ≠ Downstream Dependency Resolved.

Agent Execution Stopped ≠ Agent Authority Terminated.

Partial Retirement ≠ Complete Retirement.

Residual Dependency Identified ≠ Residual Dependency Resolved.

Retirement Plan Completed ≠ Retirement Outcome Verified.

Administrative Retirement ≠ Operational Finality.

Retirement Closure ≠ Elimination of Historical Responsibility.

Canonical Boundary

Software Retirement Standard (SRTS) governs the controlled transition of

identifiable software objects from active, supported, maintained, deployed, or otherwise operational conditions toward defined retirement states.

It establishes the governance requirements for retirement identification,

justification, planning, end-of-support management, migration, dependency resolution, operational withdrawal, verification, residual obligations, and closure.

SRTS does not require every software retirement to include replacement or migration.

It does not establish that support termination automatically ends operational use.

It does not treat artifact removal as proof that all dependencies have been eliminated.

It does not replace specialized data, security, authority, or operational governance mechanisms.

Its principal governance output is the:

Software Retirement Governance Profile

This output establishes the evidence-supported relationship between the

software being retired, its intended retirement state, the activities performed, the operational consequences, and the obligations that remain.

Canonical Closing Statement

Software does not cease to matter merely because an organization stops maintaining, distributing, or using it.

An unsupported application may remain essential to an operational process.

A retired library may remain embedded in an active product.

A discontinued interface may still serve external consumers.

A withdrawn software service may retain data that must remain accessible.

An autonomous agent may stop executing while delegated permissions or unresolved tasks continue to exist.

And a firmware version may remain installed in physical equipment long after its official support period has ended.

SRTS establishes the governance structure required to distinguish

retirement decisions from retirement reality.

It requires software withdrawal to remain connected to the identity,

purpose, dependencies, operational role, and support conditions of the retiring object.

It governs the transition of functions, users, data, and dependencies where replacement or migration is necessary.

It preserves the distinction between ending support, ending maintenance,

ending execution, and achieving operational retirement.

It requires residual dependencies and obligations to remain identifiable

rather than disappearing behind an administrative retirement status.

And it establishes retirement closure only through an evidence-supported determination of the actual retirement state and the conditions that remain governed.

The retirement of software is not simply the end of its execution. It is the controlled termination or transfer of its operational role, dependencies, authority, and continuing obligations.

Software Retirement Standard (SRTS) transforms software retirement from a

discontinuation event into a verifiable lifecycle transition that preserves operational integrity, dependency accountability, and governance continuity beyond the software's active life.

Module Architecture

→ Retirement Planning Module (RPM)

→ End-of-Support Management Module (ESMM)

→ Software Migration Module (SMM)

→ Residual Dependency Management Module (RDMM)

→ Retirement Closure Control Module (RCCLM)

Related Documents

→ About Software Retirement Standard (SRTS)